CSCI 6461 | TOPIC 1

Introduction to Computing Systems & Binary Foundations

Abstraction, stored-program organization, data representation, and processor performance

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

Course Information & Administration

Instructor Contact

- **Email**: `jburns@ada.edu.az`
- **Office**: B317
- **Office Hours**: Tues/Thurs 12:00–14:00 (or by appointment)
- **Note**: Email is the appropriate channel for academic matters. No Teams or Blackboard messages, please.

Course Expectations

- **The Golden Rule**: Once class begins, personal conversations are not permitted.
- **Demonstrating Understanding:** You must be able to *demonstrate understanding* of work submitted for assessment. If the authenticity of your work is in doubt, you will be required to explain it in person.

AArch64 Development and Execution Environment

Software, Emulators and Hardware

- **C & aarch64 Assembly**: Source languages used to connect high-level code with AArch64 machine-level behavior.
- **QEMU**: Cross-platform full-system and user-mode emulation for AArch64.
- **Cloud VMs and Laptops**: Google Axion and Apple silicon (M-series).

Online Resources (Simulators)

- **WebAssembliss**: Browser-based aarch64 assembly environment.
- **CPUlator**: Interactive ARMv7 simulator used where appropriate for architectural comparison.

Reference Platform

Examples use AArch64 as the primary reference. Selected exercises use AArch32 only when a comparison clarifies an architectural difference.

Topic 1 Outline

1. **Architecture Foundations & Abstraction**
   - Computer architecture, abstraction levels, ISA, and microarchitecture
2. **Stored-Program Organization & Memory**
   - Stored-program execution, memory organization, hierarchy, and the memory wall
3. **Binary Representation & Arithmetic**
   - Binary and hexadecimal, endianness, two's complement, extension, and condition flags
4. **Quantitative Processor Performance**
   - Latency, throughput, CPI, the Iron Law, speedup, Amdahl's Law, and benchmarking

SECTION 1

Architecture Foundations & Abstraction

What Is Computer Architecture?

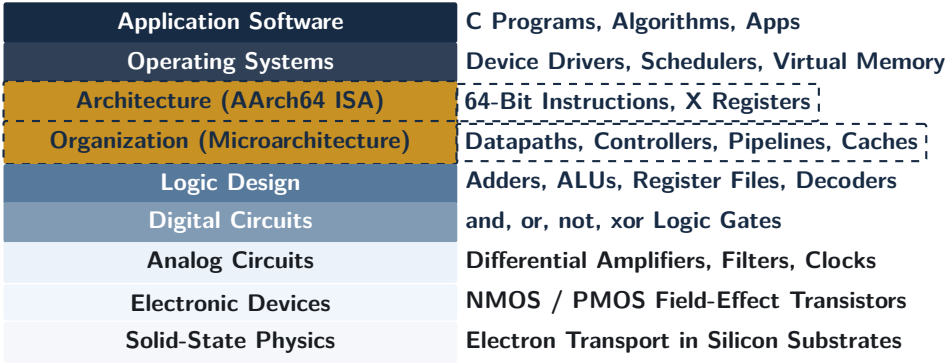
- Connects **software-visible behavior** with **hardware implementation**.
- Provides a reliable interface that software can target without knowing underlying physics.
- Implementation choices strongly influence performance, efficiency, and practical hardware limits.
- Design is a balancing act between correctness, performance, power, area, and cost.
- The ISA provides the key contract between software and the processor that executes it.

Course Approach

Design approach: RISC principles. **Reference ISA:** AArch64.

The 9 Levels of Computer Abstraction

Computers manage immense complexity by enforcing clean hierarchical interfaces:



When Abstractions Leak

Software View

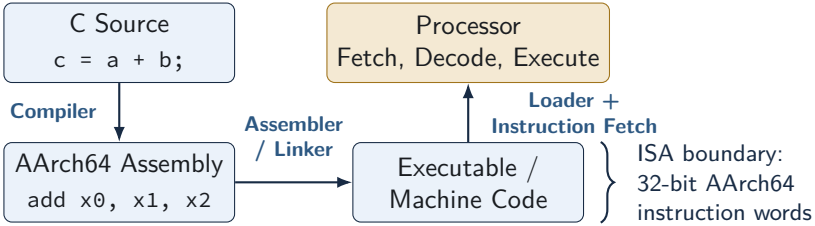
- Variables, arrays, branches
- Function calls, data types
- Dynamic memory allocation

Architectural Effect

- Register allocation, cache misses
- Branch prediction, stack frames
- Overflow, sign extension, page faults

Performance-sensitive software often exposes lower-level machine behavior. Programmers may need to consider locality, branch behavior, and data layout when execution time matters.

From Source Code to Execution



The compiler and assembler translate program intent into AArch64 instructions. The processor then fetches and executes those instructions according to the ISA, while the microarchitecture determines how that execution is implemented internally.

ISA and Microarchitecture

Instruction Set Architecture

- Instruction encodings & behavior
- Registers `x0`–`x30`, `sp`, PSTATE
- Memory, exception behavior, and privilege levels

Microarchitecture

- Pipeline organization, superscalar width
- In-order or out-of-order execution
- Branch prediction, arithmetic latency

Different processors can implement the exact same ISA using radically different microarchitectures, allowing legacy software to remain portable and functional across multiple generations of continuous hardware scaling.

RISC Design Principles

- **Load/Store Architecture:** Memory is accessed exclusively through specific load and store instructions, keeping complex arithmetic isolated to registers.
- **Simplified Decoding:** Fixed-length instruction encodings simplify instruction fetch, alignment, and decode.
- **Abundant Registers:** A large, uniform register file minimizes the need to spill temporary variables to slower main memory.

The RISC Philosophy

RISC designs favor simple, regular instructions and rely on compilers to combine them effectively, reducing complexity in instruction decoding and execution.

AArch64 Encoding Structure

- AArch64 instructions are fixed-width **32-bit words**, strictly aligned on 4-byte boundaries in memory.
- Destination (`Rd`) and source operands (`Rn`, `Rm`) use fixed 5-bit register-number fields across most formats.
- The remaining bits encode the opcode, shift modifiers, and immediate values, ensuring the processor can map the entire operation locally.

Example: 64-Bit Addition (`add x0, x1, x2` $\Longrightarrow$ `0x8b020020`)

Encoding cleanly maps `Rd = x0`, `Rn = x1`, and `Rm = x2` within the uniform 32-bit word.

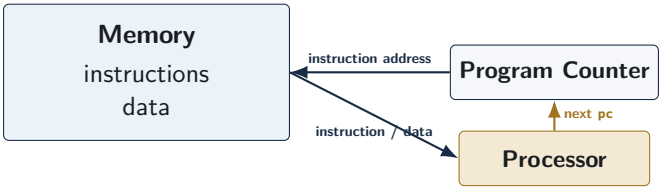
Fixed-width encoding simplifies instruction fetch, alignment, and decoding compared with variable-length instruction sets such as x86.

SECTION 2

Stored-Program Organization & Memory

The Stored-Program Concept

- A stored-program computer keeps **instructions and data in memory as binary information**.
- The processor does not inherently "know" that a memory word is code or data; interpretation depends entirely on how that address is used.
- The **Program Counter (pc)** identifies the address of the next instruction to be fetched into the execution pipeline.



The stored-program model allows a machine to run different programs by changing the instructions held in memory rather than rewiring the hardware.

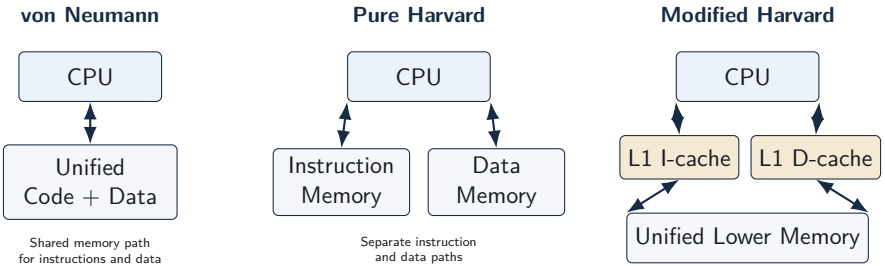
From Fetch to Completion: One Instruction

Example Instruction

```
add x0, x1, x2
```

- For ordinary sequential execution, the next instruction is fetched at the next consecutive 4-byte address.
- Branches, execution exceptions, and subroutine returns dynamically replace the ordinary sequential next-pc value.
- Real processors aggressively overlap these activities through deep pipelining to increase overall throughput.

Memory Topologies: von Neumann, Harvard, Modified Harvard



- Modern high-performance processors commonly use a **modified Harvard organization**: separate first-level instruction/data caches with a unified software-visible address space.
- The key architectural question is not just "where is memory?" but **which paths instruction fetch and data access use**.

Modified Harvard and Newly Generated Code

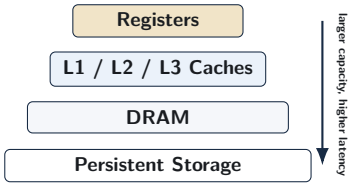
- A JIT compiler or runtime system may **generate machine instructions by writing bytes as data**.
- Those writes physically travel through the data side of the processor's memory hierarchy.
- The instruction side may still hold an older, stale copy of the same address, making it unsafe to jump blindly to the newly written bytes.

The exact AArch64 cache-maintenance and barrier instructions are system-software details that we will revisit later. Without the required synchronization, instruction fetch may observe stale instructions rather than the newly generated code.

Memory Hierarchy: Why Multiple Levels Exist

Design Tension

- Small storage can be extremely fast.
- Large storage is cheap per bit but slow.
- No single technology maximizes speed, capacity, and cost simultaneously.



Why the Hierarchy Works: Locality

Programs rarely access memory randomly. They exhibit:

- **Temporal Locality:** Recently accessed items are likely to be reused soon.
- **Spatial Locality:** Nearby memory locations are likely to be accessed soon.

The Memory Wall

- Processor execution capability has scaled much faster than off-chip DRAM latency.
- A cache hierarchy attempts to keep the processor working on nearby copies of frequently used data.
- When locality fails and a request travels down the hierarchy, dependent instructions may wait many cycles, although independent work can sometimes continue.
- While memory bandwidth scales reasonably well, physical memory *latency* remains a strict bottleneck.

Illustrative Latency Scale

Registers (1 cycle) $\rightarrow$ L1 Cache ($\sim$3 cycles) $\rightarrow$ Lower Caches (10–40 cycles) $\rightarrow$ DRAM (100+ cycles)

Exact latencies are implementation-dependent; the architectural lesson is the exponential gap between nearby and distant storage.

SECTION 3

Binary Representation & Arithmetic

Bits and Interpretive Context

- Hardware stores binary bit patterns; **meaning comes from context**.
- The stored-program model therefore permits the same memory bytes to be interpreted differently depending on how they are used.
- Hardware enforces no inherent "type safety"; the same bit pattern can be interpreted differently by different instructions.

Interpretation Perspective	**Meaning of 32-bit Word `0x8b020020`**
AArch64 Machine Instruction	`add x0, x1, x2`
Unsigned 32-bit Integer	$2{,}332{,}164{,}128_{10}$
Signed 32-bit Integer	$-1{,}962{,}803{,}168_{10}$

Bits do not carry a built-in “type”; instructions and software conventions determine the interpretation.

Positional Binary and Hexadecimal

- In positional binary, each bit contributes a power of two (Value $= \sum_{i=0}^{n-1} b_i 2^i$).
- Hexadecimal is a compact notation for binary because one hex digit represents exactly four bits.
- For fixed-width unsigned arithmetic, results are computed modulo 2^n; values beyond $2^n - 1$ wrap around within the available n bits.

Example

$$\texttt{10110110}_2 = 128 + 32 + 16 + 4 + 2 = 182_{10} = \texttt{0xb6}$$

Endianness: Mapping Multi-Byte Values to Memory

- Endianness determines which byte of a multi-byte value occupies the lowest memory address.
- In little-endian storage, the least-significant byte is placed first.

Store 32-bit value `0x12345678` starting at address `0x1000`

Little-Endian Rule (AArch64, x86)

Lowest address → least-significant byte

Big-Endian Rule (Network Protocol)

Lowest address → most-significant byte

Why Two's Complement?

- A signed representation should support positive and negative values while keeping arithmetic hardware simple.
- Two's complement provides:
  - one single representation of zero (avoiding $+0$ and -0 ambiguity),
  - the same binary adder logic for signed and unsigned addition,
  - simple subtraction through addition of a negated operand.

Signed Interpretation

For an n-bit two's-complement number, Value $= -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$.

Give the most-significant bit a negative weight, and add the remaining positive bit weights normally.

Constructing a Negative Two's-Complement Number

Example: Represent -5 in 8 Bits

$+5 = \texttt{00000101}$

Invert all bits: $\texttt{11111010}$

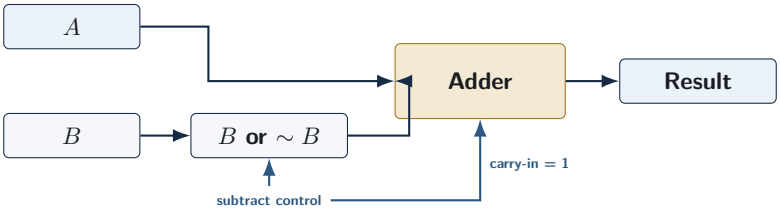
Add 1: $\texttt{11111011}$

Therefore, $-5 = \texttt{11111011}$

- The bit pattern can also be evaluated directly $(-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5)$.
- An n-bit two's-complement integer ranges from -2^{n-1} to $2^{n-1} - 1$.
- Because the number line wraps modulo the bit-width boundary, arithmetic operations naturally overflow into correct representations without special hardware cases.

Unified Adder Datapath and Subtraction

- The same n-bit adder can operate on bit patterns interpreted as either signed or unsigned integers.
- Two's-complement negation gives $-B = \sim B + 1$, so subtraction becomes $A - B = A + (\sim B) + 1$.



A unified adder lets the processor reuse the same core arithmetic hardware for both addition and subtraction.

Sign Extension and Zero Extension

- When a smaller value is widened, the upper bits must be filled in a way that preserves the intended numeric meaning.

Signed Widening

Sign extension replicates the source sign bit.
Example: `ldrsb` preserves a negative 8-bit value when widened.

Unsigned Widening

Zero extension fills upper bits with zeros.
Example: `ldrb` preserves the unsigned magnitude of an 8-bit value.

Why It Matters

Using the wrong extension changes the interpreted value and can invalidate comparisons, bounds checks, or address calculations; in some contexts, this can contribute to software vulnerabilities.

AArch64 NZCV Condition Flags

Arithmetic instructions update status flags in the PSTATE register to describe the result characteristics:

Flag	Name	Evaluation Meaning
N	Negative	Most-significant result bit is 1
Z	Zero	All result bits are zero
C	Carry	Unsigned carry-out (add) or no-borrow (sub)
V	Overflow	Signed mathematical result is outside the destination range

Same Bits, Different Questions

For $1111_2 + 0001_2$ in 4 bits, the stored result is 0000_2.
Unsigned interpretation: $15 + 1 = 16$ produces a carry-out ($C = 1$).
Signed interpretation: $-1 + 1 = 0$ does not overflow the limits ($V = 0$).

Signed vs. Unsigned Conditional Branching

After `cmp x0, x1`, the processor has conceptually subtracted the operands and set NZCV. The compiler maps C-level types (e.g., `int` vs `unsigned int`) to specific branch mnemonics.

Branch	Context	Flag Condition	Meaning
`b.eq` / `b.ne`	Universal	$Z = 1$ / $Z = 0$	Equal / Not Equal
`b.lt` / `b.ge`	Signed	$N \neq V$ / $N = V$	Less Than / Greater or Equal
`b.lo` / `b.hs`	Unsigned	$C = 0$ / $C = 1$	Lower / Higher or Equal

Why the Mnemonic Depends on Interpretation

The same 64-bit pattern can represent a negative signed integer or a very large unsigned integer. The correct branch condition follows the program's intended interpretation, not the raw bits alone.

SECTION 4

Quantitative Processor Performance

Defining Performance: Latency and Throughput

- For a fixed workload, Performance $= \frac{1}{\text{Execution Time}}$.
- **Latency:** time required to complete one isolated task.
- **Throughput:** number of tasks completed per unit time.
- Some applications are strictly latency-bound (e.g., high-frequency trading), while others are throughput-bound (e.g., batch video encoding).

Latency: one task takes 5 cycles

Throughput: one task may finish each cycle

IPC (Instructions Per Cycle) is a measure of instruction throughput inside a processor, but it is not a universal measure of overall application-task throughput.

Dynamic Instruction Count and Average CPI

- **Dynamic Instruction Count (IC):** the actual number of instructions executed during runtime for a particular run.
- **Cycles Per Instruction (CPI):** workload-average cycles per retired instruction ($\text{CPI} = \frac{\text{Total Clock Cycles}}{\text{Dynamic Instruction Count}}$).
- Different instructions, cache misses, dependencies, and branch mispredictions can contribute different execution delays.
- Aggressive compiler optimizations often lower IC, but may inadvertently increase CPI if the remaining instructions are highly memory-dependent.

CPI Is Not a Fixed Property of the Processor

A processor will naturally exhibit wildly different CPI values on different programs, data inputs, and memory-access patterns.

The Iron Law of Processor Performance

The Fundamental CPU Performance Equation

$$T_{\text{CPU}} = \text{Instruction Count (IC)} \times \text{CPI} \times T_{\text{clk}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clk}}}$$

Component	Influenced By	Examples
IC	Algorithm, compiler, ISA	instruction selection, code structure
CPI	Pipeline, dependencies, memory hierarchy	stalls, speculation, cache misses
T_{clk}	Circuit paths, pipeline organization	critical-path delay, implementation choices

Important Trade-Offs

These factors are **not independent**. A design change that improves one factor can often worsen another (e.g., deeper pipelines increase clock speed but also increase stall CPI).

Iron Law: Comparative Evaluation Example

Machine A

- IC $= 1.0 \times 10^9$
- CPI $= 2.0$
- $f_{\text{clk}} = 4.0$ GHz

$$T_A = \frac{1.0 \times 10^9 \times 2.0}{4.0 \times 10^9} = \mathbf{0.50}\ \text{s}$$

Machine B

- IC $= 1.2 \times 10^9$
- CPI $= 0.8$
- $f_{\text{clk}} = 2.5$ GHz

$$T_B = \frac{1.2 \times 10^9 \times 0.8}{2.5 \times 10^9} = \mathbf{0.384}\ \text{s}$$

$$\text{Speedup} = \frac{T_A}{T_B} = \frac{0.50}{0.384} \approx \mathbf{1.30}\times$$

The Iron Law equation strictly compares measured assumptions; the numerical results alone do not prove which microarchitectural feature inherently caused the difference.

The Megahertz Myth and Speedup

- A higher clock frequency does not guarantee better overall performance because IC and CPI heavily alter execution time.
- Pure frequency scaling is largely constrained today by power dissipation limits (the end of Dennard scaling).
- **Speedup** compares old and new execution times:

$$\text{Speedup} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{\text{Performance}_{\text{new}}}{\text{Performance}_{\text{old}}}$$

- If runtime drops from 10 s to 4 s:

$$\text{Speedup} = \frac{10}{4} = 2.5\times$$

Amdahl's Law: Why Partial Optimization Is Limited

- Suppose only fraction f of an application's execution time can be accelerated through architectural enhancements.
- Even if the enhanced fraction became infinitely fast, the unaffected portion $(1 - f)$ would still limit overall speedup.

Amdahl's Law Equation

$$S_{\text{overall}} = \frac{1}{(1 - f) + \frac{f}{S_{\text{enhanced}}}}$$

Worked Example

For $f = 0.60$ and $S_{\text{enhanced}} = 10$:

$$S_{\text{overall}} = \frac{1}{0.40 + 0.06} \approx 2.17\times$$

Limiting Case

If that 60% became infinitely fast:

$$S_{\max} = \frac{1}{0.40} = 2.5\times$$

Benchmarking and Architectural Trade-Offs

- Simple metrics such as MIPS (Millions of Instructions Per Second) are unreliable across different ISAs because instruction counts and instruction meanings drastically differ.
- Benchmark suites such as SPEC CPU provide standardized workloads for comparing complete systems.
- Geometric means are commonly used to summarize normalized performance ratios across a benchmark suite.

Trade-Offs Preview

Later topics will quantify how choices such as pipeline depth, issue width, and cache capacity can improve one performance factor while simultaneously increasing complexity, latency, power, or another Iron-Law term.

Review and Self-Test Questions

Topic 1 Summary: Architecture & Memory

- **Abstraction Layers:** Systems span from high-level software down to physical devices, with the ISA forming the key software/hardware contract.
- **Stored-Program Model:** Instructions and data are represented uniformly as bits in memory; pc directs instruction fetch.
- **Modified Harvard Organization:** Separate first-level instruction/data paths can efficiently coexist with a unified software-visible address space.
- **Memory Hierarchy:** Multiple storage levels systematically exploit temporal and spatial locality to drastically reduce average access time.

Core Takeaway

System software must respect these boundaries; for example, newly generated code may require explicit synchronization between data writes and instruction fetch.

Topic 1 Summary: Representation & Performance

- **Two's Complement:** Signed integers use a representation that supports one zero and a unified hardware addition/subtraction datapath.
- **Condition Flags:** NZCV correctly distinguishes unsigned carry boundaries from signed arithmetic overflow limits.
- **The Iron Law:** CPU execution time depends jointly and inextricably on instruction count, average CPI, and clock period.
- **Amdahl's Law:** Overall speedup is mathematically limited by the fraction of execution time that remains untouched by enhancements.

Core Takeaway

Microarchitectural improvements involve trade-offs: improving one performance factor can increase complexity, power, latency, or pressure elsewhere in the system.

Self-Test Diagnostic Questions: Architecture & Binary

1. **ISA vs. Microarchitecture:** Categorize the following:
   - Number of visible registers (`x0`–`x30`).
   - Size of the L1 instruction cache.
   - Binary machine encoding of the add instruction.

2. **Two's Complement Construction:** Show the mathematical logic and bitwise inversion steps required to reliably encode -37 as an 8-bit two's-complement value.

3. **Carry vs. Overflow:** In a 4-bit ALU evaluating $1001_2 + 1100_2$, determine the output sum, Carry flag (C), and Signed Overflow flag (V). Explain why they differ.

Self-Test Questions: Performance & Memory

1. **Iron Law Analysis:** A compiler optimization decreases dynamic instruction count by 20%, but increases average CPI by 15%. Does overall application performance improve? Calculate the exact speedup.

2. **Amdahl's Law Application:** Memory stalls strictly account for 70% of application execution time. In order to achieve an overall hardware speedup of $2.5\times$, what exact speedup must be targeted in the memory portion alone?

3. **Generated Code and Split Caches:** Explain precisely why a runtime system that dynamically writes new machine instructions requires barrier synchronization before those instructions are safely fetched and executed.

Looking Ahead

Next Lecture Preview

Our Next Topic: RISC vs. CISC: Instruction Set Design Principles & Trade-offs

- **Complexity Distribution:** Examining how different instruction sets divide work between software and hardware.
- **Key Architectural Trade-offs:** Evaluating how instruction encoding, decode complexity, pipeline behavior, and register use affect processor performance.
- **Historical Context:** Understanding why x86 dominates desktop computing while AArch64 and ARM dominate the mobile and efficiency-first sectors.